PHASE 3 DOCUMENT

ComicCraft: AI Comic Story Creator using Gemini Models

Phase Name: Gemini Model Integration and Core Backend Development

Project Name	ComicCraft: AI Comic Story Creator using Gemini Models
Phase	Phase 3 – Gemini Integration and Backend
Project Type	Generative AI Comic Creation Application
Main Technology	Google Gemini Models
Prepared By	_____
Guide / Faculty	_____
Date	_____

Academic Project Phase Documentation

1. Phase Overview

This is Phase 3 of **ComicCraft: AI Comic Story Creator using Gemini Models**. This phase implements the secure backend connection between ComicCraft and Gemini models. It focuses on API configuration, protected credential management, authenticated generation requests, comic-specific prompt construction, structured response processing, safety controls, and the first working AI story-generation service.

2. Phase Objective

The objective of this phase is to allow a logged-in ComicCraft user to submit a comic idea and receive a structured, editable comic draft. The backend accepts the story settings, securely sends an appropriate request to Gemini, receives the generated content, validates and formats the output, saves the draft when required, and returns it to the creator interface. The Gemini API key remains protected on the server at all times.

3. Backend Service Setup

A backend service is created with a suitable framework such as Node.js with Express or Python with FastAPI/Django. The service receives requests from the ComicCraft frontend, checks user authentication, validates story settings, manages comic projects, communicates with Gemini models, and stores approved draft data. The backend can connect to a database for user accounts, projects, generation records, story content, and revision history.

4. Secure API Credential Management

The Gemini API key is stored only in a protected server environment variable or managed secrets service. The key must not be placed in frontend source code, client-side browser storage, screenshots, shared documents, mobile application bundles, or public GitHub repositories. During development, the secret can be kept in a local environment file that is excluded from version control. During deployment, secure hosting-platform configuration should be used.

5. Core Endpoint Design

The backend may provide protected endpoints such as **POST /api/comics/generate** for initial comic creation, **POST /api/comics/{id}/regenerate** for regenerating a selected section, **GET /api/comics/{id}** for retrieving an owned comic project, and **PUT /api/comics/{id}** for saving user edits. Each endpoint verifies the active user session and ensures that a creator can access only their own projects unless explicit sharing is later enabled.

6. Request Validation

Before requesting AI content, the backend validates the comic idea, selected genre, tone, intended audience, comic length, and requested generation mode. It rejects empty or malformed input, limits unnecessarily large requests, confirms that the user is authenticated, and applies rate controls. Friendly messages should explain invalid conditions, for example: “Please enter a comic idea” or “Select a genre before generating the story.”

7. Comic Prompt Construction

The backend transforms the user's settings into a controlled Gemini prompt. The prompt can request a title, premise, character list, scene outline, and panel-by-panel script in a predictable structure. It may instruct Gemini to create original characters, avoid copyrighted franchises, keep the requested tone appropriate for the selected audience, and include visual descriptions, dialogue, narration, captions, and sound effects only where relevant. Prompt templates make generated content more consistent and easier for the application to display.

8. Generation Workflow

1. Creator logs in and opens Create New Comic.
2. Creator enters a story idea and chooses settings.
3. The frontend sends the request to the protected backend.
4. The backend verifies the user, validates the request, and applies usage rules.
5. The backend builds a comic-generation prompt.
6. The request is sent securely to Gemini.
7. Gemini returns story and panel content.
8. The backend extracts, validates, and formats the response.
9. A comic draft is stored under the creator's project.
10. The frontend displays the resulting story for review and editing.

9. Structured Response Processing

To support a usable editor, ComicCraft should request content in a structured format. The backend can map the generated result into fields such as title, logline, genre, characters, scenes, page number, panel number, panel visual description, dialogue, narration, caption, and sound effect. If Gemini returns incomplete or improperly formatted content, the backend should attempt a safe fallback, display the usable portion, or request a regeneration rather than storing an invalid draft.

10. Content Safety Controls

The backend applies safety-focused input and output handling. It should block or safely refuse prohibited requests, discourage copying protected characters or named living artists' styles, and apply age-appropriate content controls based on application policies. The system should tell users that generated output is AI-assisted and should be reviewed for originality, factual claims, and suitability. A reporting option can be planned for content that violates the platform rules.

11. Error Handling

The backend must handle missing data, invalid authentication, network interruption, Gemini API errors, unavailable service, quota limits, timeouts, malformed responses, and database failures. It should return simple, useful messages instead of technical errors. For example: "Comic generation is temporarily unavailable. Please try again later." Existing saved drafts must be preserved if a new request fails.

12. Security Controls

Security in this phase includes HTTPS communication, authenticated API requests, authorization checks for comic ownership, server-side API-key storage, input validation, output encoding, rate limits, restricted CORS configuration, and secure error logging. Only authorized users can retrieve or edit a project. The system should minimize storage of raw prompts and retain generation data only as needed for the creator's project and service operations.

13. Initial Feature Implementation

The initial working feature is **Generate Comic Outline**. A creator enters an original story idea and selects basic settings. ComicCraft sends a structured prompt to Gemini and displays a generated title, premise, character overview, story outline, and short panel script. The creator can then save the result as a project draft. This confirms that the full secure path—from user request to Gemini response to saved comic content—is operational.

14. Testing Plan

Testing includes protected environment-variable checks, Gemini connectivity tests, valid and invalid input tests, authentication checks, project-ownership tests, prompt-template tests, response-structure tests, safety and refusal tests, API failure tests, quota/timeout tests, and confirmation that secrets are not exposed in frontend files or repository history. Test ideas should include short concepts, detailed concepts, blank input, unsupported requests, and content that requires a safe refusal.

15. Phase Deliverables

The Phase 3 deliverables are a configured backend service, protected Gemini credential setup, authenticated comic-generation endpoint, validation rules, comic prompt templates, structured response parser, draft-saving mechanism, error handling, safety controls, initial Generate Comic Outline feature, and integration-test evidence.

16. Planned Next Phase

The next phase will be **Phase 4: Comic Creation Interface and Story Editing Features**. It will create the user-facing dashboard, comic-idea form, story settings, generated-outline view, character and scene editing tools, panel-script editor, and saved-project management interface.

17. Conclusion

Phase 3 establishes the technical foundation for AI-powered comic generation in ComicCraft. By routing Gemini requests through a protected backend, validating user input, enforcing content and ownership controls, and converting responses into editable comic fields, the project gains a secure and functional base for the creator experience developed in the next phase.

Phase Approval

Prepared by	_____
Guide / Faculty	_____
Approved by	_____